

Worksheet 2: Parallel Programming for Shared Memory Systems

This worksheet is all about mastering concurrency, exploring both the high-level OpenMP API and low-level C++ threads.

We are tackling these two topics in "reverse" order. Why? Because I firmly believe in saving the best for last. As someone who genuinely loves low-level plumbing, this means you will get to build your own concurrent systems with native C++ threads in the second half of the worksheet.

Part 1: OpenMP

We begin with OpenMP, focusing on parallelizing the heat equation solver we have been building in the lectures. As we discussed, OpenMP is designed to make parallel programming highly accessible and that is exactly what this worksheet aims to demonstrate.

But don't be fooled by its apparent simplicity. You will start by parallelizing the solver framework, and yes, it really will only take a few lines of code. However, because I know you crave a bit more action, we aren't stopping there: we will extend the matrix-free implementation with an implicit time-stepping scheme, which means rolling up our sleeves to implement an iterative solver.

To keep things interesting, we will also ingest our code into our benchmarking framework. You will analyze performance, optimize your implementation, and perhaps most importantly learn how to dodge classic OpenMP traps.

Fun Fact: If you aren't aware of these performance pitfalls, it is entirely possible to write "parallel" code that actually runs slower than your original sequential version. While it's still fun to implement, convincing your boss it was worth the effort might be a tough sell!

Part 2: C++ Threads

Now, for the absolute fun part. In the final task, we will implement a thread pool from scratch. A thread pool is a fundamental concept in concurrent programming. Building one yourself will give you a deep, under-the-hood understanding of how high-level parallel libraries like OpenMP actually manage resources. By the end of this worksheet, you won't just know how to use parallelism, you will know how to build the infrastructure that powers it.

Curious? I know I am. Let's dive in!

Getting Started

Once opening the project folder in VSCode you should be prompted to reopen it in the container. If not, you can do this manually by clicking on the green icon in the bottom left corner of VSCode and selecting "Reopen in Container". After a short moment the container environment should be ready. Please note, that in case you have not yet setup

the container image or anything changed in the docker file this might take a bit longer.

You find detailed installation instructions in the README provided in the project's root folder. If you run into any trouble, just ask — we are happy to help!

Code Structure

The project consists of five main folders:

- **applications/** – Contains the executable for the heat equation solver. Feel free to add applications if you want to experiment with your own code.
- **include/i2php/** – Header files for your classes and logic.
- **source/** – The corresponding implementation (.cpp) files.
- **benchmarks/** – Contains benchmarks to evaluate the performance of your implementations.
- **tests/** – Contains unit tests which you can run to verify the correctness of your implementations. See the README for more details. Also feel free to add your own tests. Writing tests is a great way to deepen your understanding of the code. You might wanna have a look at test-driven development and GoogleTest for this.

Submission

Please submit your solution until Wednesday, July 1st, 2026 11:59 PM by opening a merge request of your developing branch into the main branch of the repository. The merge request should include the following:

- The modified code files including all required changes for the worksheet.
- A short PDF document summarizing the performance analysis of your implementations, including the benchmark results and your reasoning about the observed trends. In addition, include the answers to the guiding questions provided in the worksheet, along with any relevant observations or insights you gained during the implementation and testing process.

Today's Worksheet Tasks

The worksheet consists of six tasks. The first five are intended to be completed in the given order. While some tasks might be partially independent, working through them sequentially is definitely the easiest and most structured approach. The last task which is the previously mentioned thread pool task can be worked on independently of the other tasks. If you prefer to start with the thread pool task, feel free to do so.

We begin with the parallelization of the linear algebra vector class using OpenMP in Task 1. Task 2 builds on this by parallelizing the matrix class. For both tasks, you will use the provided benchmarking framework to analyze the performance of your implementation.

This will give you early insights into what can go wrong when OpenMP directives are misplaced or when inappropriate directives are chosen. The results may still be correct, but poor performance can quickly become a major source of frustration.

Building on these findings, we move on to parallelizing the matrix-based and matrix-free heat equation solvers. Again, performance analysis will be part of the process, with a particular focus on the choice of scheduling strategy and chunk size.

In the last task dealing with OpenMP, we extend the heat equation solver with an implicit time-stepping scheme and implement the Jacobi method—naturally, parallelized with OpenMP. By this point, you might be getting tired of the benchmarking framework (unlikely, but not impossible). So, for this task, we will not require a deep performance analysis. Instead, rely on the experience you have gained in the earlier tasks. Of course, if you are still having fun with benchmarks (and who could blame you?), feel free to dive deeper—bonus points await!

Finally, in task 6 you will implement a thread pool from scratch using native C++ threads. This task is completely independent of the previous ones, so you can work on it at any time. We will not apply the thread pool to our heat equation solver but test it using the Monte Carlo method for estimating π on which we worked on in the fourth exercise. The focus in this task is on the implementation itself, but if you have the time and interest, you can also perform a performance analysis using the provided benchmark for estimating π with and without the thread pool.

When working with benchmarks, please include the results in your report to support your explanations (screenshots are sufficient). Additionally, report the details of the CPU used for benchmarking. Especially important are the number of cores, whether hyperthreading is enabled, and if applicable, the number of performance and efficiency cores.

As always, you can use the tests provided in the directory `tests/` to verify the correctness of your implementation. Note that these tests do not check whether your code runs in a multithreaded fashion. You do not need to explicitly set the number of threads in your implementation. Let OpenMP use its current default. Both the benchmarks and tests are configured to adjust the number of threads according to their needs.

One last important note regarding the implementation: Please make sure to follow best practices by using `default(none)` in your OpenMP pragmas and explicitly specifying which variables are shared, private, or otherwise throughout your code. This approach not only promotes clear and maintainable code but also helps prevent subtle, hard-to-debug issues related to variable scoping in parallel regions.

With that being said, let's start parallelizing our heat equation solver!

Hint:

When working with the benchmarks, you can selectively run only those tests that are relevant to you by using filtering. To do this, pass the flag

```
--benchmark_filter=<filter>
```

When executing the benchmark, replacing `<filter>` with the desired substring to match benchmark names. For example, using `Addition` as the filter will run all benchmarks whose names contain the word *Addition*, while skipping all others.

Task 1: Linear Algebra Vector (6 Points)

We begin the worksheet by parallelizing our linear algebra vector class. There is not too much to say about this task—it's straightforward and a great way to warm up.

Parallelize the Implementation

Use OpenMP to parallelize the following member functions and overloaded operators:

- `number dot_product(const LaVector<number, N>& other) const;`
- `LaVector<number, N> operator+(const LaVector<number, N>& vec) const;`
- `LaVector<number, N>& operator+=(const LaVector<number, N>& vec);`
- `LaVector<number, N> operator-(const LaVector<number, N>& vec) const;`
- `LaVector<number, N>& operator-=(const LaVector<number, N>& vec);`
- `LaVector<number, N> operator*(number num) const;`
- `LaVector<number, N>& operator*=(number num);`
- `LaVector<number, N> operator/(number num) const;`
- `LaVector<number, N>& operator/=(number num);`
- `number l2_norm_squared() const;`
- `number norm_l2() const;`
- `number norm_l1() const;`
- `number norm_linf() const;`
- `LaVector<number, N> schur_product(const LaVector<number, N>& other) const;`

Think about which scheduling type and chunk size make sense for each of these cases—and which ones might just slow things down for no good reason. Briefly justify your decisions. You don't need to dive deep into benchmarking yet—just use the knowledge you obtained from the lecture and the script. We will analyze performance more thoroughly later on.

Don't be surprised if you only need to add a few lines of code. As mentioned earlier: OpenMP is meant to be easy — and parallelizing our serial implementation really is that easy. So enjoy the simplicity of writing parallel code with OpenMP ;)

To check the correctness of your implementation, you can rely on the provided tests. Just keep in mind that they verify correctness of results only—they won't tell you whether your code actually uses multiple threads.

Performance Analysis

Once you've finished the parallelization, it's time for a quick performance sanity check. The benchmark suite in `benchmarks/linear_algebra/la_vector_benchmark.cpp` runs the code automatically with different thread counts. Take a look at the results: do you always see the speedups you expect? If not, don't worry—it's all part of the game. Try to come up with some explanations for why increasing the number of threads doesn't always lead to better performance. (Spoiler alert: memory access patterns, memory transfer time, overhead, and load imbalance often might play a role. This also holds for subsequent tasks.)

Task 2: Linear Algebra Matrix (8 + 2 Points)

Having parallelized our linear algebra vector class, we now continue with the matrix class. While the task is conceptually similar, it requires more careful decisions to ensure good performance. A poorly chosen parallelization strategy may result in the use of multiple threads but still perform worse than the serial version. For this reason, we will rely on the provided benchmarking framework to assess performance.

Parallelize the Implementation

As a first step, we will parallelize the implementation without worrying about performance tuning—yet. Use OpenMP to parallelize the following member functions and overloaded operators:

- `number trace() const;`
- `LaMatrix<number> operator+(const LaMatrix<number> &mat) const;`
- `void operator+=(const LaMatrix<number> &mat);`
- `LaMatrix<number> operator-(const LaMatrix<number> &mat) const;`
- `void operator-=(const LaMatrix<number> &mat);`
- `LaMatrix<number> operator*(const LaMatrix<number> &mat) const;`
- `LaVector<number> operator*(const LaVector<number> &vec) const;`
- `LaMatrix<number> operator*(number num) const;`
- `void operator*=(number num);`
- `LaMatrix<number, N> operator/(number num) const;`
- `void operator/=(number num);`

To keep things manageable, we will not use nested parallel regions. If you are unsure which loop to parallelize in a nested structure, a good rule of thumb is to start with the outermost loop.

Matrix-Matrix Multiplication Benchmarking

With the initial parallelization complete, we now turn to performance analysis. Our first focus is the matrix-matrix product, implemented by the overloaded operator

```
i2phpp::LaMatrix<number>  
i2phpp::LaMatrix<number>::operator*(const LaMatrix<number> &mat) const;
```

The benchmark provided in `benchmarks/linear_algebra/la_matrix_benchmark.cpp` can be used to evaluate the performance for different thread counts. Investigate:

- Which loop(s) should be parallelized for best performance?
- Does loop collapsing improve performance?
- How do CPU time and wall-clock time compare?

Summarize your findings and discuss how different implementation choices influence the observed speedups. While you should keep the order of nested loops as provided in the code skeleton for your submitted implementation, feel free to experiment with different loop orders on your own. Exploring alternative loop orderings can provide valuable insights into how factors like memory access patterns affect performance—and sometimes even lead to surprising results. So, be curious and explore!

Matrix-Matrix Addition/Subtraction Benchmarking

Next, we evaluate the performance of matrix addition/subtraction, using the following function as an example

```
void  
i2phpp::LaMatrix<number>::operator+=(const LaMatrix<number> &mat);
```

As before, we perform the runtime measurements with the provided benchmarks in the file `benchmarks/linear_algebra/la_matrix_benchmark.cpp`. Determine the optimal loop to parallelize and whether loop collapsing is beneficial. Once again, summarize and explain your observations.

Compare the achieved speedups with the matrix-matrix multiplication case. Are there notable differences? What might explain these differences?

Note: If you wish, you can adjust the matrix size used in the benchmarks. By default, the matrices are square, and their size is determined by the global variable `size` (see line 18 of the benchmark file). Once you determine the optimal configuration for `operator+=`, you may use the same strategy for the related operations `+`, `-`, `-=`, and scalar multiplications.

Bonus: Analyze the Performance of the Matrix-Vector Product (+2 Points)

Analyze the performance of the matrix-vector product implemented in

```
i2phpp::LaVector<number>
i2phpp::LaMatrix<number>::operator*(const LaVector<number> &vec) const;
```

For measuring the runtime, use the corresponding benchmarking framework provided in the file `benchmarks/linear_algebra/la_matrix_benchmark.cpp` to evaluate performance. Summarize your findings and compare them to those from the matrix-matrix multiplication and addition/subtraction cases. What similarities or differences do you observe? How can they be explained?

Task 3: Matrix-Free Heat Equation Solver (8 Points)

Having parallelized the linear algebra matrix and vector classes, we are almost ready to run a fully parallel matrix-based heat equation solver. While setting up the system matrix is still done sequentially, the time-stepping loop is already (almost) fully parallelized—except for one missing piece: the function

```
la_vector_type
i2phpp::HeatOperator<dim, number, la_vector_type,
↳ la_matrix_type>::source_term_vector(number current_time) const;
```

Your first goal in this task is to parallelize this source term function using OpenMP.

With that done, the matrix-based solver will be fully parallel in its time stepping. Next, it's time to tackle the matrix-free implementation. Take a look at the corresponding branch `heat_data.operation_type == OperatorType::matrix_free` in the function `advance_time_step(...)` of the heat operator, and apply all necessary changes to parallelize the matrix-free solver using OpenMP.

While parallelizing the matrix-free implementation, experiment with different scheduling schemes and chunk sizes. Use the corresponding benchmarks provided in the file `benchmarks/heat_operator/heat_operator_benchmark.cpp` to guide your experiments. Before running benchmarks, take a moment to predict which scheduling schemes and chunk sizes might work best, then compare your expectations with the actual results. Discuss why you observe similarities or differences. How does the problem size—that is, the number of degrees of freedom—affect your scheduling decisions?

Finally, compare the CPU and wall-clock times measured for both the matrix-based and matrix-free implementations. What differences do you notice, and how can you explain them?

Task 4: System Matrix Initialization (8 Points)

In this task, we will finally parallelize the setup of the system matrix. The logic for assembling the system matrix can be found in the free function


```
template <int dim, typename number, typename la_matrix_type>
void
setup_finite_difference_heat_equation_system_matrix(
const HeatData<dim, number> &heat_data,
const DoFHandler<dim, number> &dof_handler,
la_matrix_type &system_matrix);
```

You can find this function in the header file of the heat operator which is located in `include/heat_equation/heat_operator.hpp`. Corresponding benchmarks are available in the file `benchmarks/heat_operator/heat_operator_benchmark.cpp`.

Begin by discussing whether synchronization is necessary in your parallel implementation and explain your reasoning. Then, use the benchmarks to measure the runtime for the three scheduling types: `static`, `dynamic`, and `guided`. For each scheduling type, experiment with different chunk sizes and report your observations. Explain why one scheduling type might outperform the others in certain situations, and vice versa.

Keep in mind that correctness of your implementation can be verified using the matrix-based heat operator tests, which rely on a properly assembled system matrix to produce correct results.

Task 5: Implicit Time Stepping with the Jacobi Method (10 + 3 Points)

I must admit, we've done a lot of benchmark analysis so far but written only a few lines of code. Don't you miss the excitement of designing new logic from scratch—starting with a serial implementation, debugging it, then parallelizing and hunting parallel bugs? I definitely do! So, for this task, the focus is on coding rather than benchmark analysis—I promise!

Theory

Until now, we have only used explicit time stepping for our heat equation solver, which limits the maximum stable time step size. In this task, we will add an implicit time stepping scheme that allows significantly larger time steps. To keep things simple, the theoretical discussion will focus on the 1D case, but extending to 2D or 3D is straightforward. Recall the one-dimensional heat equation

$$\frac{\partial u(x, t)}{\partial t} = \frac{\partial}{\partial x} \left(\kappa(x) \frac{\partial u(x, t)}{\partial x} \right) + q(x, t), \quad (1)$$

where $u(x, t)$ is the temperature, $\kappa(x)$ is thermal diffusivity, and $q(x, t)$ is a source term. Discretizing space at points x_i and time at t^n , the finite difference formulation becomes

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{\kappa_{i+1} - \kappa_{i-1}}{2\Delta x} \frac{u_{i+1}^{n+1} - u_{i-1}^{n+1}}{2\Delta x} + \kappa_i \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} + q_i^n, \quad (2)$$

where subscripts denote spatial position and superscripts denote time steps. Rearranging leads to the matrix-vector form

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t \mathbf{K} \mathbf{u}^{n+1} + \mathbf{q}^{n+1}. \quad (3)$$

Here, the unknown $\mathbf{u}^{n+1}$ appears on both sides, so we must solve a linear system.

Many solvers exist for such systems, from direct methods like Gaussian elimination to iterative ones like conjugate gradient. For this task, we will implement the Jacobi method—an iterative solver that’s relatively simple to parallelize. Instead of solving in one step, iterative methods start with an initial guess and refine it repeatedly until convergence.

Rearranging equation (3), we write

$$(\mathbf{I} - \Delta t \mathbf{K}) \mathbf{u}^{n+1} = \mathbf{u}^n + \mathbf{q}^{n+1}, \quad (4)$$

which matches the standard form $\mathbf{Ax} = \mathbf{b}$ with

$$\mathbf{A} := \mathbf{I} - \Delta t \mathbf{K}, \quad (5)$$

$$\mathbf{x} := \mathbf{u}^{n+1}, \quad (6)$$

$$\mathbf{b} := \mathbf{u}^n + \mathbf{q}^{n+1}. \quad (7)$$

With that we continue our discussion considering the introduced standard form $\mathbf{Ax} = \mathbf{b}$. The Jacobi update formula for the k -th iteration is given by

$$x_i^{(k+1)} := \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n, \quad (8)$$

starting from an initial guess $\mathbf{x}^0$. To decide when to stop iterating, we use the relative residual

$$r = \frac{\|\mathbf{Ax} - \mathbf{b}\|_2}{\|\mathbf{x}\|_2}, \quad (9)$$

with the Euclidean norm $\|\cdot\|_2$. When r falls below a given tolerance, we stop. To avoid infinite loops, we also limit the iteration count to a maximum of 1000.

Implementation

With the theory laid out above, we are now ready to proceed with the implementation. The code skeleton is prepared inside the `advance_time_step(...)` function in the branch `heat_data.operator_type == OperatorType::matrix_free_implicit`. Note that the yet empty function

```
number
i2php::HeatOperator<dim, number, la_vector_type, la_matrix_type>::
jacobi_update(const std::size_t      index,
const number      time,
const number      current_time_increment,
const la_vector_type &old_iter_vector,
const la_vector_type &rhs) const;
```

and the `matrix_free_stencil(...)` function used by the explicit matrix-free solver might ease your implementation (but you don’t have to use them). The tolerance for the stopping criterion is given by `HeatData::iterative_solver_tolerance`. Please do

not use a matrix-based but a matrix-free approach for your implementation.

Feel free to implement the Jacobi method as you like, but it should be parallelized with OpenMP. If you are unsure how to parallelize, the insights from previous tasks will help.

A test is provided in `tests/heat_tests.cpp` to verify correctness. As before, tests check correctness but not whether multiple threads are used.

With all that said, have fun implementing! :)

Bonus: Benchmark Analysis (+3 Points)

Perform a performance analysis similar to Task 3's explicit matrix-free implementation: experiment with different OpenMP scheduling policies and chunk sizes. What do you observe? Explain the results, focusing on both CPU and wall-clock times. Are there similarities or differences compared to the explicit case? Trace these observations back to your implementation details to understand their causes.

Task 6: Thread Pool (15 + 3 Points)

Let's turn to the final and, from an implementation perspective, most exciting part of this worksheet—at least in my opinion! In this task, we will delve into thread pools, a fundamental concept widely used in concurrent programming.

When working with native C++ threads, many applications require creating and joining threads repeatedly. Take our heat equation solver as an example: if we were to parallelize it using standard C++ threads, we would have to spawn and join threads every single time we perform a matrix-vector multiplication, meaning at least once per time step. When simulating over millions of time steps, this constant creation and destruction introduces massive overhead, especially if the computational work per thread is relatively small. To solve this, we need a strategy that allows us to create a set of threads exactly once and then reuse them by continuously feeding them new tasks. This is precisely the purpose of a thread pool.

In this worksheet, we won't be injecting our thread pool directly into the heat equation solver. Instead, we will apply it to the same problem we already worked on in the C++ threads exercise: estimating the value of π using the Monte Carlo method. The goal of this task is to give you the chance to build a low-level thread pool completely from scratch, which will deepen your understanding of how high-level parallel libraries like OpenMP manage resources under the hood. By the end of this task, you won't just know how to use parallelism, you will understand how to build the infrastructure that powers it.

You'll find a code skeleton for a thread pool in `/include/i2phpp/tools/thread_pool.hpp`, along with essential member variables. Before we proceed to fill the skeleton with code, let's briefly examine these member variables:

- **workers:** This is a `std::vector` of threads that will execute the tasks submitted to the thread pool. These threads are created upon the thread pool's construction and are destroyed when the thread pool object is destructed.

- **jobs:** This member, of type `std::queue<std::function<void()>`, acts as the task manager for work submitted to the thread pool. Whenever a task needs to be performed, it's pushed onto this queue and removed once a worker thread picks it up. As a `std::queue`, it operates on a First-In, First-Out (FIFO) principle, ensuring tasks are processed by worker threads in the same order they were added. The function signature `std::function<void()>` indicates that tasks submitted to the thread pool must be functions that return `void` and don't take any arguments.
- **queue_mutex:** This mutex will be used to synchronize access to shared resources among the worker threads, such as the `jobs` queue itself.
- **queue_condition:** A condition variable used to notify worker threads about special events related to the job queue (e.g., when new jobs become available).
- **work_done_condition:** Another condition variable, specifically for implementing a synchronization function. Its exact usage will become clearer when you implement the corresponding synchronization mechanism later.
- **active_jobs:** A helper variable that tracks the number of threads currently busy with executing a job.
- **stop_thread_pool:** This boolean helper variable is used to signal worker threads to cease waiting for new jobs and terminate their execution. You'll see its precise function as you work through the implementation.

These are all the necessary class members for our thread pool implementation; you will not need to add any new ones during this task. Now, let the fun of implementation begin!

Constructor

Let's begin by implementing the constructor for our thread pool. It's declared as:

```
explicit ThreadPool(const std::size_t num_threads =  
↳ std::thread::hardware_concurrency());
```

Here, `num_threads` specifies how many threads our thread pool should manage, which consequently dictates the number of threads we need to create within this constructor. You're already familiar with thread creation, but the key question now is: what task should we assign to these threads?

Essentially, each thread should repeatedly:

1. Check if there's a task waiting in the `jobs` queue.
2. If yes, retrieve the task and execute it.
3. If no, wait (in a sleep state) for a new task to become available.

Let's break down the details of this core worker loop. First, to enable continuous checking for tasks, we'll use a `while(true)` loop. This loop should only terminate under specific conditions: when the member variable `stop_thread_pool` is `true` AND there are no more

tasks left in the `jobs` queue.

Once you've implemented the mechanism for exiting the loop, proceed to implement how threads retrieve tasks from the queue. The function `std::queue::pop()` will be useful here. Remember to use proper synchronization to prevent race conditions when multiple threads try to access the shared `jobs` queue concurrently.

So far, we've covered how threads stop working and how they take tasks. The next question is: when should a thread actually attempt to take a task? Continuously polling (repeatedly checking if the queue is empty) would be a waste of resources. A more efficient solution is to put the worker thread to sleep until an external event—for example, a user adding a new task to the queue—wakes it up. This is precisely what we want to achieve using a condition variable. Ensure you correctly handle spurious wakeups to prevent threads from waking unnecessarily.

And that's it! These are all the requirements for our thread pool's constructor.

Destructor

So far, we've focused on creating threads, but we haven't addressed their proper destruction. The threads instantiated in the constructor need to be gracefully shut down using `std::thread::join()`. This is the responsibility of the thread pool's destructor.

Before you can `join()` the threads, you must ensure they terminate correctly. This means they need to break out of their continuous loops. As discussed, you'll achieve this by setting the `stop_thread_pool` variable to `true`. Additionally, you must notify any threads that are currently sleeping (waiting for tasks) so they can wake up, check the `stop_thread_pool` flag, and exit their loops.

Always remember to check for any synchronization requirements during this process to avoid race conditions. Once all threads are properly signaled to terminate and have indeed exited their loops, you can safely `join()` them. With that, your destructor's task will be complete.

Dealing with New Tasks

Naturally, our thread pool needs a way for users to submit new tasks. For this, you'll implement the `enqueue` function:

```
void enqueue(std::function<void()> job);
```

The `job` argument is the task the user wants the thread pool to execute, so it needs to be added to the `jobs` queue. Remember that since the queue is a shared resource, you'll might need synchronization to ensure correctness.

Furthermore, simply adding a job isn't enough. You must notify your worker threads about the new task. If any threads are currently sleeping, this notification will wake them up, allowing a free thread to pick up and execute the new task.

Synchronization

There is one last function we need to implement before we can use the tests to verify our thread pool implementation. This function is crucial for ensuring that all tasks have been completed before we proceed with certain operations. Don't worry, we'll guide you through its implementation:

```
void ThreadPool::wait_for_all() const;
```

This `wait_for_all()` function acts as a synchronization barrier. It should only return when two conditions are met: the thread pool's task queue is empty, and all worker threads in the pool are idle (i.e., none of them are currently executing a task and are instead waiting for new jobs to arrive).

Let's implement the function body of `wait_for_all()`. There isn't too much to do here. Essentially, you need to put the current thread to sleep until two conditions are simultaneously true: the `jobs` queue is empty, and the `active_jobs` counter (which tracks the number of threads currently working on a job) is zero. For the condition variable, use the `work_done_condition` class member. That's all for the function body!

However, we're not quite done. We still need to maintain the `active_jobs` count and notify the `wait_for_all()` function when all jobs are complete. This needs to be done within the worker thread's loop (which you implemented in the constructor). Specifically:

1. Increment the `active_jobs` variable whenever a worker thread starts executing a new job.
2. Decrement `active_jobs` as soon as a thread finishes its job.
3. After a thread completes its job and decrements `active_jobs`, it should check if the `jobs` queue is empty and if `active_jobs` is now zero. If both conditions are true, it should wake up any thread currently sleeping in the `wait_for_all()` function.

And that's it! Great job! You've successfully implemented a basic thread pool class!

Testing the Thread Pool

Once your thread pool implementation is complete, it is time to verify that everything works as expected. To support this process, we provide a set of tests using different number of threads in `tests/estimating_pi_test.cpp`. Rather than checking the thread pool implementation directly, these tests validate the correctness of the Monte Carlo π estimation that relies on your thread pool.

Two tests are available. The first serves as a sanity check and runs the Monte Carlo estimation without using the thread pool. This ensures that the underlying algorithm behaves correctly on its own. The second test executes the same computation using your thread pool implementation. If your implementation is correct, both tests should pass successfully. If the second test fails while the first one succeeds, this is a strong indication that the issue lies within the thread pool itself. The test results can therefore provide valuable guidance when debugging and refining your implementation.

Benchmarking the Thread Pool (Not mandatory (0 Points))

As mentioned earlier, the primary objective of this task is to give you hands-on experience with the design and implementation of a low-level thread pool. For this reason, a detailed performance analysis is not required. Nevertheless, exploring the performance characteristics of your implementation can be both insightful and rewarding. So if you have the time and interest, feel free to dive into the benchmarking of your thread pool implementation.

You can use the benchmark provided in `benchmarks/tools/estimating_pi_benchmark.cpp`. This benchmark compares the runtime of the Monte Carlo π estimation with and without the use of the thread pool, allowing you to observe the practical impact of parallel execution.

As you examine the results, consider where performance differences might originate and what aspects of the implementation contribute to them. Thinking about task scheduling, synchronization overhead, and workload distribution can provide valuable insights into the behavior of parallel systems. As a starting point, it is recommended to focus on wall-clock time rather than CPU time.